

DA3408 Assignment-2

K. Sree Hasini DA24BO41

Note: Please find all the evidences asked in the evidence.md file

Question 1

- Naive image size - 868MB, Multi-stage image size - 611MB
- Percentage(%) reduction = $\frac{868-611}{868} \times 100 = 29.60\%$
- The main difference comes from the Python libraries required. While training the model (before the API stage), we only require pandas, scikit-learn and joblib, but not uvicorn and fastapi.
- Whereas for the later phase (for inference), **pandas is not required**.
- Using `--no-cache-dir` during pip installation prevents pip from storing its package download cache inside the image, which helps reduce the final image size.
- The dataset (CSV) and `model.joblib` are already put in `.dockerignore` for both naive and multi-stage builds.
- A slim version of Python is used as the base image for both cases, because the comparison is valid only when the environments are identical.

Question 2

- Docker Compose allows different parts of an application, like the API and Redis cache, to run in separate containers, making them easier to manage, debug, and update independently.
- It defines these services, their networking, dependencies, and configurations in a single `docker-compose.yml` file.
- Kubernetes solves a larger problem by orchestrating containers across a cluster of multiple machines, providing scheduling, scaling, self-healing, and load balancing.
- **Docker Compose manages separate application components on a single host, while Kubernetes manages containerized applications across a cluster.**

Question 3

- I assign each pod a CPU request and limit of 1 CPU, since each pod is responsible for validating one CSV shard and is therefore given a dedicated CPU for its computation.
- **Total available CPU:** 2 nodes $\times$ 2 CPUs/node = 4 CPUs
- **Maximum pods running simultaneously:** $\frac{4 \text{ CPUs}}{1 \text{ CPU/pod}} = 4 \text{ pods}$
- Therefore:

parallelism = 4,	completions = 8
------------------	-----------------
- This allows 4 pods to validate 4 shards at a time. Once they finish, the next 4 pods validate the remaining 4 shards. A higher parallelism would leave extra pods in **Pending**, while a lower value would leave some CPU capacity unused.

Part 3

- Minikube's default **standard** StorageClass uses a **hostPath** provisioner. This means the stored data is kept on the local disk of a particular node, rather than on a storage system shared by all nodes.
- So, when different pods run on different nodes, they may not be able to access files written by pods on other nodes. Therefore, a shared volume is not suitable for collecting results in a multi-node Job.
- The Kubernetes API avoids this issue because each pod's logs can be retrieved regardless of which node the pod ran on.

Part 4

- **Total available CPU:** 3 nodes $\times$ 2 CPUs/node = 6 CPUs
- **Maximum pods that can run simultaneously:** $\frac{6 \text{ CPUs}}{1 \text{ CPU/pod}} = 6 \text{ pods}$
- Therefore, I would increase the parallelism from 4 to:

parallelism = 6,	completions = 8
------------------	-----------------
- This allows 6 shards to be processed in the first wave and the remaining 2 shards in the second wave, making use of all available CPU capacity.

Question 4

Part 2

- The **ReplicaSet controller** is responsible for replacing the deleted pod.
- The Deployment manages the ReplicaSet and specifies the desired number of replicas as **replicas: 2**.
- When one pod is deleted, the number of running pods becomes: $2 - 1 = 1$
- The ReplicaSet detects that only 1 pod is running instead of the desired 2, so it creates a new pod.
- Thus, the number of running pods is restored to: $1 + 1 = 2$

Part 3

- A **Job** is designed to run a task until it **completes**, so it is not suitable for a continuously running API.
- Unlike a **Deployment**, a Job does not continuously maintain a desired number of running replicas or provide rolling updates.
- If the application needs to be updated, a Job would need to be replaced with a new Job rather than gradually updating the existing pods.
- Therefore, a continuously running API should use a **Deployment** instead of a Job.

Part 4

- A **Deployment** is designed for **long-running applications**. It maintains the desired number of replicas, replaces failed pods, and supports rolling updates.
- A **Job** is designed for **batch workloads**, where pods perform a specific task and terminate successfully after the task is completed.
- Therefore, the **spam-detection API** is suited to a Deployment because it needs to continuously serve requests.
- The **shard-validation workload** is suited to a Job because each shard needs to be processed once, after which the workload should terminate.